SSD1306 OLED Display Driver & Graphics Subsystem

1. Overview of SSD1306 Display Controller

The **SSD1306** is a single-chip CMOS OLED/PLED driver with controller for organic / polymer light-emitting diode dot-matrix graphic display systems. In this project, a **0.96-inch 128x64 Monochrome OLED module** is used.

---

2. Technical Specifications & Memory Architecture

Parameter	Specification
Resolution	128 columns $\times$ 64 rows = 8,192 dots (pixels)
Operating Voltage	3.3V to 5V (Internal Charge Pump generates OLED drive voltage $\approx 7\text{V}-9\text{V}$)
Interface	I2C (2-Wire)
I2C Address	7-bit: `0x3C` (STM32 HAL 8-bit shifted: `0x78` for Write, `0x79` for Read)
GDDRAM Size	128 bytes $\times$ 8 pages = **1,024 bytes (1 KB)**

Graphic Display Data RAM (GDDRAM) Structure

The screen is organized vertically into **8 Pages (Page 0 to Page 7)**, each page consisting of **8 rows of pixels** across **128 columns**:

```
          Column 0 ..................................... Column 127
Page 0  | [D0..D7] (Rows 0-7)   ...   [D0..D7]
Page 1  | [D0..D7] (Rows 8-15)  ...   [D0..D7]
Page 2  | [D0..D7] (Rows 16-23) ...   [D0..D7]
...     | ...                           ...
Page 7  | [D0..D7] (Rows 56-63) ...   [D0..D7]
```

Each byte written represents a vertical slice of 8 vertical pixels, where Bit 0 is the top pixel and Bit 7 is the bottom pixel.

---

3. Custom Pure-C Driver Implementation

To conserve flash/RAM and avoid bulky third-party libraries (e.g. U8g2 or Adafruit GFX), a custom zero-dependency driver is implemented in [`Core/Src/main.c`](file:///Users/harinarayanans/Documents/Oled_display/Core/Src/main.c).

3.1 Local Framebuffer

```
static uint8_t SSD1306_Buffer[SCREEN_WIDTH * SCREEN_HEIGHT / 8]; // 1024 bytes
```

- All drawing operations occur in STM32 SRAM (fast memory manipulation).
- Rendering to the physical screen occurs only when `ssd1306_UpdateScreen()` is called, writing 8 pages (128 bytes each) to the SSD1306 over I2C.

3.2 Pixel Addressing Equation

To toggle or color a pixel at coordinate (x, y):

$$\text{Byte Index} = x + \left(\left\lfloor\frac{y}{8}\right\rfloor \times 128\right)$$

$$\text{Bit Mask} = 1 \ll (y \pmod 8)$$

```
void ssd1306_DrawPixel(int x, int y, uint8_t color) {
    if (x < 0 || x >= 128 || y < 0 || y >= 64) return;
    if (color == 1) {
        SSD1306_Buffer[x + (y / 8) * 128] |= (1 << (y % 8));   // Pixel ON
    } else if (color == 0) {
        SSD1306_Buffer[x + (y / 8) * 128] &= ~(1 << (y % 8));  // Pixel OFF
    } else {
        SSD1306_Buffer[x + (y / 8) * 128] ^= (1 << (y % 8));   // Inverse / XOR
    }
}
```

3.3 Bresenham's Line Algorithm

The driver implements **Bresenham's line algorithm** (**`ssd1306_DrawLine`**) to draw arbitrary diagonal, vertical, and horizontal dividing borders with integer arithmetic, avoiding floating-point operations.

3.4 5x7 Monospace Font & Scaling

- Stored in array **`font5x7[95][5]`** covering ASCII 32 (**`' '`**) to 126 (**`'~'`**).
- Scaling support:
- **`size = 1`**: 6×8 pixels per character (normal menu and status text).
- **`size = 2`**: 12×16 pixels per character (used for large alarm headers **`"MEDICINE!"`** and **`"WRONG PILL!"`**).

---

4. Potential Evaluator Questions & Answers

- **Q: Why use a 1024-byte framebuffer in MCU RAM instead of writing directly to the display?**
- *A: The SSD1306 memory architecture is organized in 8-pixel vertical pages. To change a single pixel without disturbing the other 7 vertical pixels on that column, you would need to read the byte over I2C, mask it, and write it back. Having a local 1 KB buffer in STM32 SRAM allows instantaneous drawing and transfers whole pages in single burst transactions.*
- **Q: Can the STM32F103C8T6 afford 1024 bytes of RAM for the display?**
- *A: Yes. The STM32F103C8T6 has 20 KB (20,480 bytes) of SRAM. A 1024-byte buffer consumes only 5% of total available RAM, leaving 95% free for stack, heap, and peripheral structures.*
- **Q: How does the display generate ~8V for the OLEDs from a 3.3V input?**
- *A: The SSD1306 incorporates an internal DC-DC charge pump circuit enabled during initialization using command **`0x8D, 0x14`** and external charge pump capacitors connected across pins **`C1P/C1N`** and **`C2P/C2N`** on the breakout board.*